An Analysis of Actor critic algorithm

Actor critic algorithm

Actor critic algorithm -- Part 1

γ j (∑ k = 0 n − 1 γ k R j + k + γ n V π θ (S j + n) − V π θ (S j)) {\textstyle \gamma ^{j}\left(\sum _{k=0}^{n-1}\gamma ^{k}R_{j+k}+\gamma ^{n}V^{\pi _{\theta }}(S_{j+n})-V^{\pi _{\theta }}(S_{j})\right)} : TD(n) learning.

Actor critic algorithm -- Part 2

γ j (R j + γ V π θ (S j + 1) − V π θ (S j)) {\textstyle \gamma ^{j}\left(R_{j}+\gamma V^{\pi _{\theta }}(S_{j+1})-V^{\pi _{\theta }}(S_{j})\right)} : TD(1) learning.

Actor critic algorithm -- Part 3

γ j ∑ n = 1 ∞ λ n − 1 1 − λ ⋅ (∑ k = 0 n − 1 γ k R j + k + γ n V π θ (S j + n) − V π θ (S j)) {\textstyle \gamma ^{j}\sum _{n=1}^{\infty }{\frac {\lambda ^{n-1}}{1-\lambda }}\cdot \left(\sum _{k=0}^{n-1}\gamma ^{k}R_{j+k}+\gamma ^{n}V^{\pi _{\theta }}(S_{j+n})-V^{\pi _{\theta }}(S_{j})\right)} : TD(λ) learning, also known as GAE (generalized advantage estimate). This is obtained by an exponentially decaying sum of the TD(n) learning terms.

Actor critic algorithm -- Part 4

Variants Asynchronous Advantage Actor-Critic (A3C): Parallel and asynchronous version of A2C. Soft Actor-Critic (SAC): Incorporates entropy maximization for improved exploration. Deep Deterministic Policy Gradient (DDPG): Specialized for continuous action spaces.

Actor critic algorithm -- Part 5

γ j ∑ j ≤ i ≤ T (γ i − j R i) − b (S j) {\textstyle \gamma ^{j}\sum _{j\leq i\leq T}(\gamma ^{i-j}R_{i})-b(S_{j})} : the REINFORCE with baseline algorithm. Here b {\displaystyle b} is an arbitrary function.

Actor critic algorithm -- Part 6

Critic In the unbiased estimators given above, certain functions such as V π θ , Q π θ , A π θ {\displaystyle V^{\pi _{\theta }},Q^{\pi _{\theta }},A^{\pi _{\theta }}} appear. These are approximated by the critic. Since these functions all depend on the actor, the critic must learn alongside the actor. The critic is learned by value-based RL algorithms. For example, if the critic is estimating the state-value function V π θ (s) {\displaystyle V^{\pi _{\theta }}(s)} , then it can be learned by any value function approximation method. Let the critic be a function approximator V φ (s) {\displaystyle V_{\phi }(s)} with parameters φ {\displaystyle \phi } . The simplest example is TD(1) learning, which trains the critic to minimize the TD(1) error: δ i = R i + γ V φ (S i + 1) − V φ (S i) {\displaystyle \delta _{i}=R_{i}+\gamma V_{\phi }(S_{i+1})-V_{\phi }(S_{i})} The critic parameters are updated by gradient descent on the squared TD error: φ ← φ − α ∇ φ (δ i) 2 = φ + α δ i ∇ φ V φ (S i) {\displaystyle \phi \leftarrow \phi -\alpha \nabla _{\phi }(\delta _{i})^{2}=\phi +\alpha \delta _{i}\nabla _{\phi }V_{\phi }(S_{i})} where α {\displaystyle \alpha } is the learning rate. Note that the gradient is taken with respect to the φ {\displaystyle \phi } in V φ (S i) {\displaystyle V_{\phi }(S_{i})} only, since the φ {\displaystyle \phi } in γ V φ (S i + 1) {\displaystyle \gamma V_{\phi }(S_{i+1})} constitutes a moving target, and the gradient is not taken with respect to that. This is a common source of error in implementations that use automatic differentiation, and requires "stopping the gradient" at that point. Similarly, if the critic is estimating the action-value function Q π θ {\displaystyle Q^{\pi _{\theta }}} , then it can be learned by Q-learning or SARSA. In SARSA, the critic maintains an estimate of the Q-function, parameterized by φ {\displaystyle \phi } , denoted as Q φ (s , a) {\displaystyle Q_{\phi }(s,a)} . The temporal difference error is then calculated as δ i = R i + γ Q θ (S i + 1 , A i + 1) − Q θ (S i , A i) {\displaystyle \delta _{i}=R_{i}+\gamma Q_{\theta }(S_{i+1},A_{i+1})-Q_{\theta }(S_{i},A_{i})} . The critic is then updated by θ ← θ + α δ i ∇ θ Q θ (S i , A i) {\displaystyle \theta \leftarrow \theta +\alpha \delta _{i}\nabla _{\theta }Q_{\theta }(S_{i},A_{i})} The advantage critic can be trained by training both a Q-function Q φ (s , a) {\displaystyle Q_{\phi }(s,a)} and a state-value function V φ (s) {\displaystyle V_{\phi }(s)} , then let A φ (s , a) = Q φ (s , a) − V φ (s) {\displaystyle A_{\phi }(s,a)=Q_{\phi }(s,a)-V_{\phi }(s)} . Although, it is more common to train just a state-value function V φ (s) {\displaystyle V_{\phi }(s)} , then estimate the advantage by A φ (S i , A i) ≈ ∑ j ∈ 0 : n − 1 γ j R i + j + γ n V φ (S i + n) − V φ (S i) {\displaystyle A_{\phi }(S_{i},A_{i})\approx \sum _{j\in 0:n-1}\gamma ^{j}R_{i+j}+\gamma ^{n}V_{\phi }(S_{i+n})-V_{\phi }(S_{i})} Here, n {\displaystyle n} is a positive integer. The higher n {\displaystyle n} is, the more lower is the bias in the advantage estimation, but at the price of higher variance. The Generalized Advantage Estimation (GAE) introduces a hyperparameter λ {\displaystyle \lambda } that smoothly interpolates between Monte Carlo returns (λ = 1 {\displaystyle \lambda =1} , high variance, no bias) and 1-step TD learning (λ = 0 {\displaystyle \lambda =0} , low variance, high bias). This hyperparameter can be adjusted to pick the optimal bias-variance trade-off in advantage estimation. It uses an exponentially decaying average of n-step returns with λ {\displaystyle \lambda } being the decay strength.

Actor critic algorithm -- Part 7

The actor-critic algorithm (AC) is a family of reinforcement learning (RL) algorithms that combine policy-based RL algorithms such as policy gradient methods, and value-based RL algorithms such as value iteration, Q-learning, SARSA, and TD learning. An AC algorithm consists of two main components: an "actor" that determines which actions to take according to a policy function, and a "critic" that evaluates those actions according to a value function. Some AC algorithms are on-policy, some are off-policy. Some apply to either continuous or discrete action spaces. Some work in both cases.